Arquitectura AWS y Flujo de Ejecución

Diagrama de referencia de la infraestructura y del recorrido de un documento, de principio a fin. Para la explicación conceptual (qué hace el sistema, sin código) ver `COMO_FUNCIONA.md` ; para el recorrido exacto con referencias a archivo:línea, ver `TRAZA_CODIGO_COMPLETA.md` .

> Versión PDF de este documento (mismos diagramas, para imprimir o compartir sin depender de un visor de Markdown): `ARQUITECTURA.pdf` .

Modelo

El pipeline usa **Amazon Bedrock** con el perfil de inferencia cross-region `us.anthropic.claude-haiku-4-5-20251001-v1:0` (**Claude Haiku 4.5**) como modelo base — el desplegado hoy en dev y prod, verificado contra AWS Cost Explorer (ver `COSTOS.md`).

Cualquier modelo de Bedrock sirve, no solo Anthropic: todas las llamadas pasan por la API **Converse**, que es agnóstica del proveedor (mismo request para cualquier modelo alojado en Bedrock). Cada paso puede además usar un modelo distinto, para bajar costo donde no hace falta precisión o subirla donde sí. Todo configurable sin tocar código vía `.env.dev` / `.env.production` :

Variable	Paso	¿Visión?	Default
`BEDROCK_MODEL_ID`	Modelo base de todos los pasos	—	`us.anthropic.claude-haiku-4-5-20251001-v1:0`
`BEDROCK_CLASSIFICATION_MODEL_ID`	Clasificación (flujo estándar)	**Sí**	modelo base
`BEDROCK_PROCESSING_MODEL_ID`	Extracción single-shot (flujo estándar)	**Sí**	modelo base
`BEDROCK_SEGMENTER_MODEL_ID`	Segmentación del expediente	No	modelo base
`BEDROCK_EXTRACTOR_MODEL_ID`	Extracción por documento (la mayor parte de los tokens)	No	modelo base
`BEDROCK_RECOVERY_MODEL_ID`	Re-extracción Textract + modelo	No	modelo base

Los pasos del expediente son de **solo texto** (corren sobre OCR de Textract o texto nativo del PDF, nunca sobre imágenes), así que ahí funciona un modelo sin visión; los dos pasos del flujo estándar sí reciben el documento como imagen/PDF y requieren un modelo multimodal.

Antes de mover un paso a otro modelo, confirmar que ese modelo soporte lo que el paso usa — Bedrock no garantiza paridad entre los modelos que aloja:

Capacidad	La usa	Si no la soporta
Converse	Todos los pasos	Falla la llamada
Tool use forzado (`toolChoice`)	Extracción por documento (catálogos de 100+ campos)	Se degrada a JSON libre, menos confiable
Prompt caching (`cachePoint`)	Extracción por documento	El pipeline lo detecta y reintenta sin caché (ver nota abajo). Funciona, pero pierdes el descuento
Visión	Clasificación y extracción del flujo estándar	No aplica a modelos de solo texto
Ventana de contexto	Flujo estándar	Retunear `MAX_MODEL_DOC_BYTES` (default 3.5 MB, calibrado para ~200K tokens)

Cambiar el modelo requiere **redeploy** del ambiente correspondiente (la variable se hornea en la Lambda al desplegar), y habilitar antes el **"Model access"** de esa familia de modelo en la consola de Bedrock — paso manual por cuenta/región que no se automatiza desde CDK ni IAM.

Métodos de extracción: cuándo es solo IA, Textract, "OCR de Python" (pypdf) o código puro

Cuatro mecanismos distintos conviven en el repo para sacarle datos a un documento. No son intercambiables ni mutuamente excluyentes — un mismo documento suele pasar por dos de ellos en secuencia (p. ej. pypdf o Textract **adquieren** el texto, Bedrock lo **estructura**). Esta tabla es la vista unificada; el detalle de cada fila vive en el archivo referenciado.

#	Mecanismo	Qué hace	Dónde vive	Cuesta
1	**Solo IA** (visión)	Bedrock lee el PDF/imagen directamente como imagen — sin pasar antes por ningún OCR.	`ClassificationStep` / `ProcessingStep` del blueprint (flujo estándar, Camino B)	Bedrock (tokens de imagen)
2	**Textract** (solo, sin IA)	Textract `AnalyzeID` devuelve campos ya estructurados (taxonomía fija: `LAST_NAME`, `DATE_OF_BIRTH`, …); se mapean 1:1 a campos del catálogo vía `field_aliases`. Ningún LLM interviene.	`extractors/textract_analyze_id.py`	Solo Textract
3	**Textract + IA** (híbrido)	Textract `DetectDocumentText` saca las líneas OCR; Bedrock las interpreta y arma el JSON contra el schema del catálogo.	`extractors/textract_bedrock.py`	Textract + Bedrock (texto)
4	**"OCR de Python"** (pypdf, sin servicio externo)	`pypdf` lee la capa de texto nativa del PDF (modo `layout`) — gratis, exacto, sin llamar a nada. Es la primera opción siempre que el PDF sea digital (no un escaneo).	`pipeline/pdf_pages.py::_native_text`	$0
5	**Código puro** (sin IA, sin OCR)	Parseo determinístico: `ElementTree` sobre XML, `openpyxl` / `csv` sobre tablas, decodificación de QR, regex de normalización. Nunca hay incertidumbre de modelo.	`pipeline/xml_document.py`, `pipeline/tabular_document.py`, `pipeline/qr_recovery.py`, `pipeline/normalize.py`	$0

Por camino de ejecución (ver diagrama de flujo más abajo):

- **Camino A — Expediente** (`PEDIMENTO_ADUANAL`, `CERTIFICADO_POLIZA`, cualquier tipo `fields` — el único camino activo en prod hoy): Bedrock **nunca ve una imagen**. Cada página primero pasa por **(4) pypdf** si tiene capa de texto nativa; si no (escaneo) o el texto se ve corrupto, cae a **(3)-estilo Textract OCR** (`DetectDocumentText`, no `AnalyzeID`) **en paralelo** por página (`pdf_pages.py:187`). El texto resultante — de cualquiera de las dos fuentes — se le pasa a Bedrock (`text_extractor.py`) para estructurarlo contra el schema del catálogo: es texto siempre, jamás visión (por eso el paso de extracción del expediente puede usar un modelo sin capacidad multimodal, ver tabla de arriba). Una imagen suelta (JPG/PNG, `load_image_page`) salta pypdf por completo y va directo a Textract. Encima de esto, campos puntuales del pedimento (`PATENTE`, `NUMERO_CONSECUTIVO`, `ANIO_PEDIMENTO`, `ADUANA_DESPACHO`) se corrigen con **(5) código puro**: `qr_recovery.py` rasteriza la página (`pypdfium2`) y decodifica el QR embebido (`zxing-cpp`), sin IA ni Textract; y las referencias/montos pasan por `normalize.py` (regex determinístico).
- **Camino B — Flujo estándar** (tipos legacy DSL, hoy aparcados en `catalogs/save/` — ningún tipo activo lo usa): **(1) Solo IA** por defecto — Bedrock recibe el documento como imagen/PDF y clasifica + extrae en un solo paso, sin OCR de ningún tipo. Si el tipo declara `extraction_strategy` en su catálogo y la extracción inicial falla checks estructurales, se reintenta con **(2) Textract solo** (`textract_analyze_id`, JPEG/PNG únicamente) o **(3) Textract + IA** (`textract_bedrock`, cualquier formato) — ver `handler.py::_try_reextract`. Ningún catálogo activo hoy configura `extraction_strategy`, así que las filas 2 y 3 existen en código pero están dormidas en producción.
- **Tipos con fuente XML** (`_XML`, `xpath` en vez de `hint`) y **tipos con fuente tabular** (`_XLSX` / `_CSV`, `cell` / `column`): siempre **(5) código puro** — cero Bedrock, cero Textract, `source: "XML_EXTRACTION"` en la salida en vez de `"MODEL_EXTRACTION"`. Detalle completo: `../catalogs/README.md`.

En resumen, para el sistema tal como corre hoy (solo `PEDIMENTO_ADUANAL` / `CERTIFICADO_POLIZA` activos): **nunca es "solo IA"** — todo lo que no es XML/tabular pasa primero por pypdf o Textract antes de llegar a Bedrock. "Solo IA" (fila 1) y "Textract solo" (fila 2) son caminos reales del código, verificados y probados, pero solo se activan si se revive un tipo legacy en `catalogs/` o si un tipo declara `extraction_strategy`.

Arquitectura AWS

Todo el stack es serverless (`DocumentValidationStack-Dev` en dev, `DocumentValidationStack` en prod — dos stacks aislados y completos, ver `AMBIENTES_Y_SEGURIDAD.md`). La Step Function la crea el blueprint `@cdklabs/cdk-appmod-catalog-blueprints`; este repo aporta Upload, Enrichment y Query, además de las dos Lambdas de clasificación/procesamiento creadas desde `model-invoke-lambda`, y toda la configuración del catálogo.

Los colores de los nodos siguen la paleta oficial por categoría de servicio de AWS (la misma de los AWS Architecture Icons), para que cada pieza se identifique de un vistazo:

🟠 Compute (Lambda) · 🟢 Storage (S3) · 🔵 Database (DynamoDB) · 🩷 App Integration (SQS, EventBridge, Step Functions) · 🟣 Networking (API Gateway) · 🔴 Security (KMS) · 💙 Machine Learning (Bedrock, Textract) · ⚫ Cliente / externo

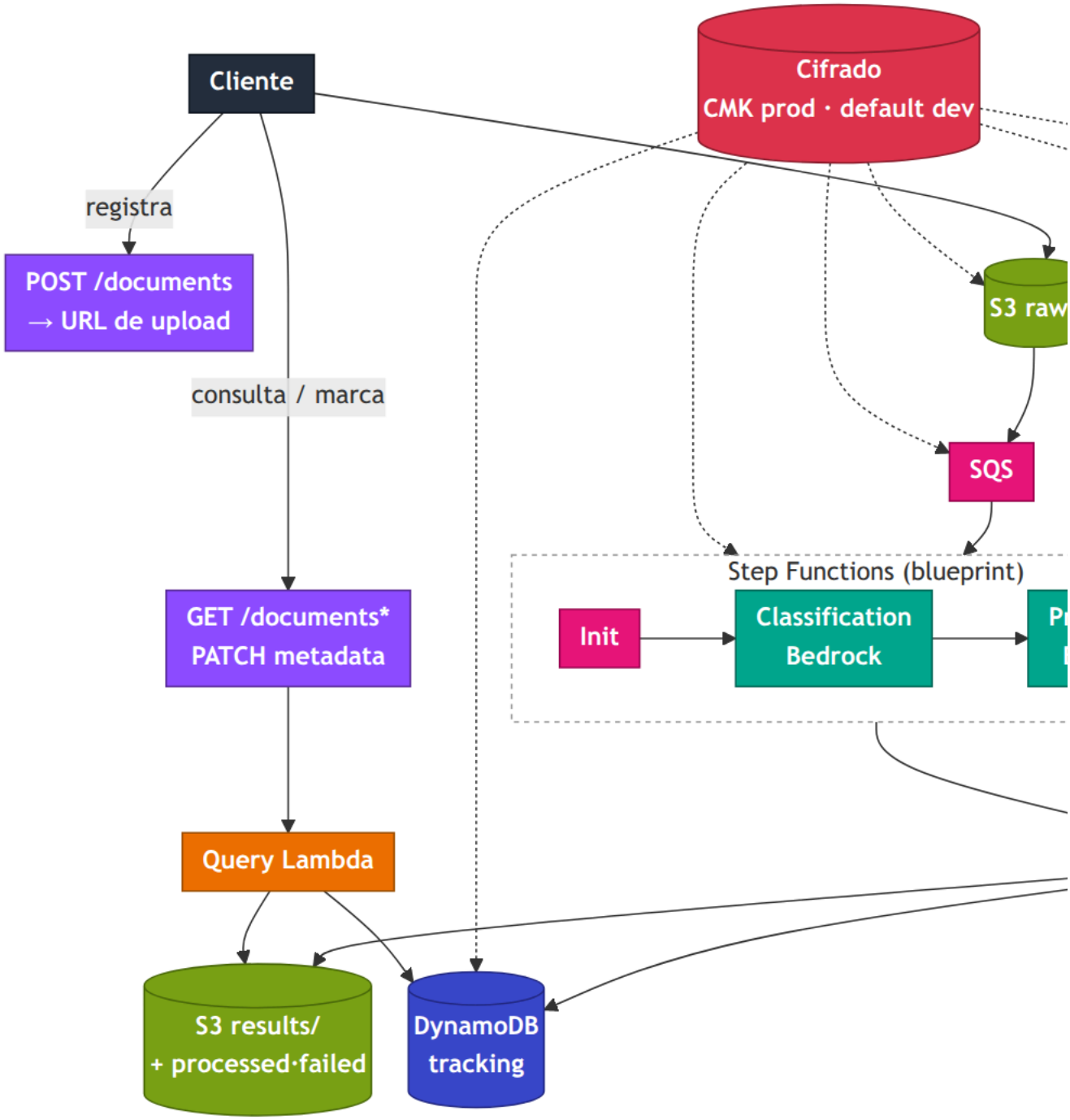


Diagrama de arquitectura AWS: Cliente registra el documento por API Gateway, sube directo a S3 raw/ con URL prefirmada, dispara SQS y Step Functions; la Enrichment Lambda escribe resultados y la Query Lambda atiende consultas y PATCH de metadata en DynamoDB.

Fuente editable: `diagrams/arquitectura-aws.mmd` .

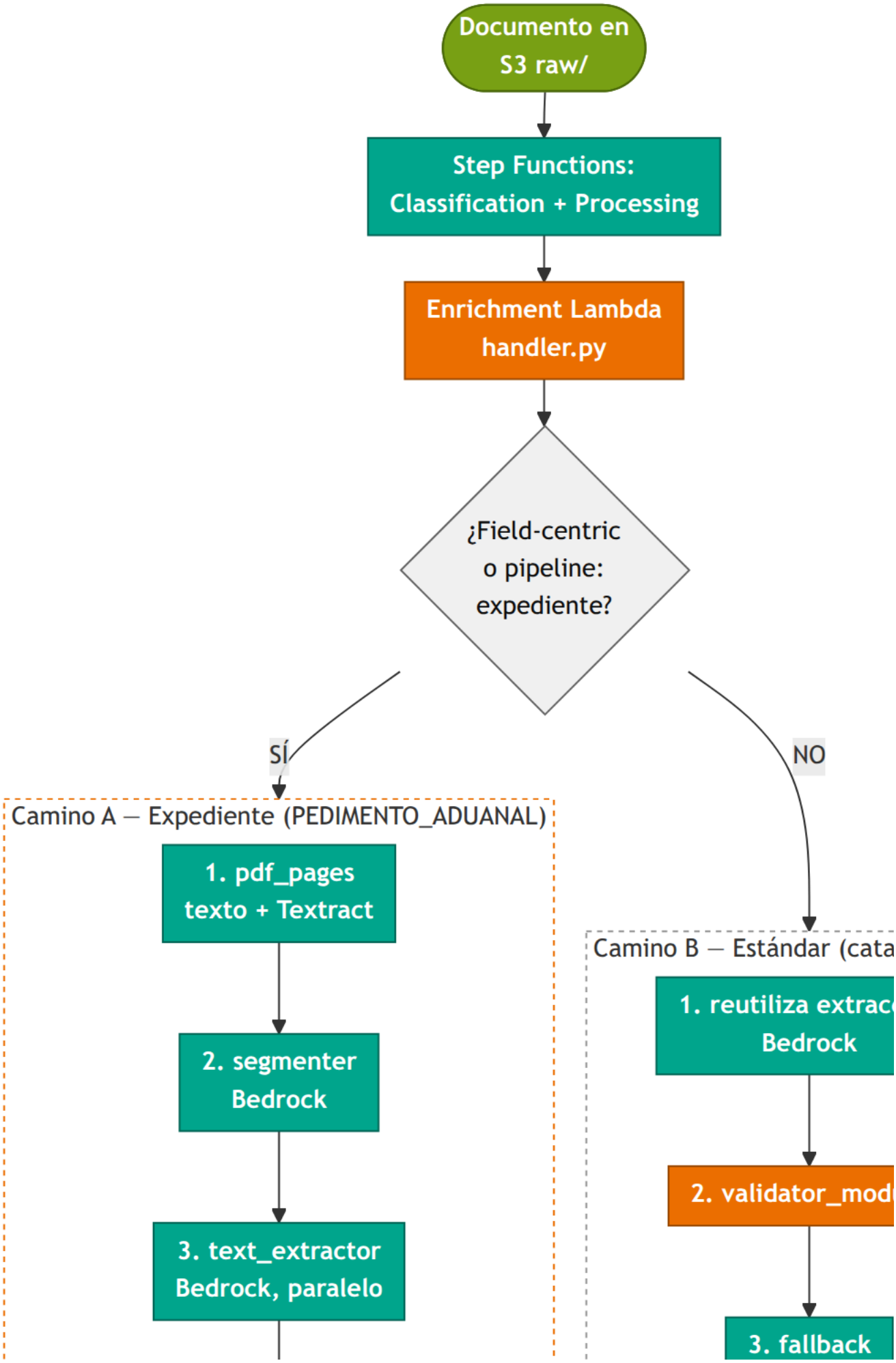
Piezas clave:

Pieza	Rol
Upload Lambda	Valida el request y devuelve una URL prefirmada; el cliente siempre escribe directo a `S3 raw/`, para cualquier tamaño. El prefijo `raw/{TIPO}/` es cómo se "fuerza" la clasificación.
S3 `raw/`	Recibir un objeto aquí dispara todo el pipeline (trigger vía SQS → Step Functions).
Step Functions	Orquestador del blueprint. La clasificación manual se resuelve sin Bedrock; la automática sí llama al modelo. `ProcessingStep` omite la llamada cuando el tipo no tiene campos en el prompt legacy y difiere la extracción al enrichment. El pipeline field-centric usa después su propia segmentación/extracción por texto.
Enrichment Lambda ⭐	El cerebro: aquí vive `handler.py` y todo `pipeline/`. Ver diagrama de flujo abajo.
Textract	OCR **solo** en páginas de PDF sin capa de texto (el resto usa pypdf, gratis).
DynamoDB catálogo	Tipos y reglas editables en caliente (`pnpm sync`), `RETAIN` — sobrevive a un `destroy` del stack. Gana siempre sobre el JSON empaquetado en runtime.
DynamoDB tracking	Envelope **compacto** por documento y columna mutable `Metadata`. `PATCH /documents/{id}/metadata` cambia solo esa columna con versión/auditoría; no altera la validación ni el snapshot de S3.
S3 `results/`	Resultado **completo**: todos los campos, arrays completos, checks y confianzas.
EventBridge (`idp-broker`)	Publica la notificación terminal (`document-processed-successful` o `document-processing-failed`) con el id y datos de origen. El resultado enriquecido se consulta por id en DynamoDB/S3.
Query Lambda	Atiende `GET /documents*` y `PATCH /documents/{id}/metadata`: lee DynamoDB/S3, aplica alcance por app y escribe únicamente la metadata externa cuando `AllowMetadataWrite` lo permite.
KMS	En prod, un CMK compartido cifra los recursos del pipeline, incluido S3. Dev usa cifrado administrado por cada servicio y no crea CMKs propios. Ver `COSTOS.md`.

Seguridad por ambiente (API keys, WAFv2, dominio propio) solo aplica a **prod**, gateada por `environment === "prod"` — detalle completo en `AMBIENTES_Y_SEGURIDAD.md`.

Flujo de ejecución

El `handler.py` de la Enrichment Lambda bifurca en dos caminos según el tipo de documento clasificado. Mismos colores por servicio que el diagrama de arquitectura: 🟠 Compute (Lambda/lógica propia) · 🟢 Storage (S3) · 🔵 Database (DynamoDB) · 🩷 App Integration (EventBridge/Step Functions) · 💙 Machine Learning (Bedrock/Textract).


Documento en S3 raw/
Step Functions: Classification + Processing
Enrichment Lambda handler.py
¿Field-centric o pipeline: expediente?
SÍ
NO
Camino A — Expediente (PEDIMENTO_ADUANAL)
1. pdf_pages texto + Textract
2. segmenter Bedrock
3. text_extractor Bedrock, paralelo
Camino B — Estándar (cata
1. reutiliza extrac Bedrock
2. validator_mod
3. fallback

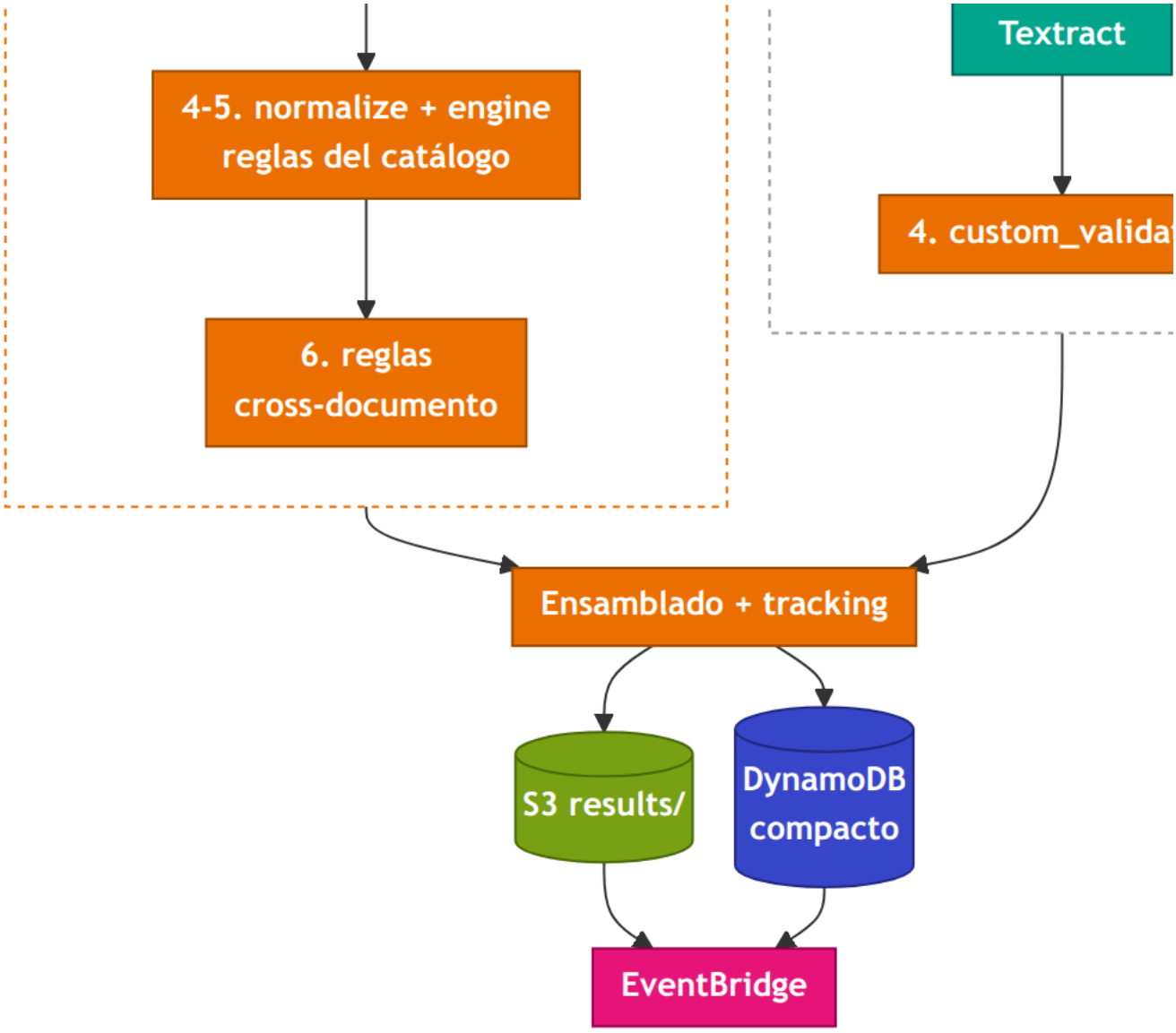


Diagrama de flujo de ejecución: un documento en S3 raw/ pasa por Step Functions, la Enrichment Lambda decide entre el Camino A (pipeline de expediente: pdf_pages, segmenter, text_extractor, normalize+engine, reglas cross-documento) o el Camino B (flujo estándar: reutiliza extracción, validator_module, fallback Textract, custom_validator); ambos producen un envelope para DynamoDB y el Camino A también persiste el resultado profundo en S3 results/.

Fuente editable: `diagrams/flujo-ejecucion.mmd`.

Notas de enrutamiento y detalle por paso (recortado del diagrama para que quepa en una vista — la traza completa está en `TRAZA_CODIGO_COMPLETA.md`):

- La condición de la izquierda decide el **camino de extracción**; `pipeline: "expediente"` en el catálogo controla además, por separado, la **candidatura a auto-segmentación** (solo tipos con ese flag se consideran cuando el PDF se sube sin declarar tipo) y la **elegibilidad para reglas cross-documento**. Detalle: `../catalogs/README.md`.
- Antes de enrichment, clasificación manual y extracción field-centric pueden hacer bypass sin llamar Bedrock. En clasificación automática sí existe una llamada upstream; el segmentador del Camino A puede corregir después esa clasificación con texto y rangos de página.
- **Camino A, paso 1**: `pdf_pages.load_pages` — texto nativo (pypdf, layout) + Textract OCR **en paralelo** solo en páginas escaneadas.
- **Camino A, paso 2**: `segmenter.segment_pages` — si el uploader declaró el tipo (`declared_type`), 1 solo segmento sin llamar al modelo; si no, Bedrock clasifica los rangos de página.
- **Camino A, paso 2.5 — selección de páginas de interés** (`page_selector.py`, opt-in por tipo): antes de extraer, cada segmento se recorta a las páginas cuyo texto cumple los marcadores que el tipo declara en `page_selection`. Es puro regex sobre texto que ya se tenía: $0 de AWS, y el ahorro cae entero del lado de Bedrock — medido, un expediente de 86 páginas subido con el tipo declarado pasa de mandar 86 páginas a mandar 16 (**81 % menos**). Solo la llamada de extracción ve las páginas filtradas; `_text` (reglas cross-documento) y la búsqueda del QR conservan el segmento completo, porque son gratis y recortarlas solo podría perder información. Si ninguna página alcanza el umbral, o la selección resulta demasiado amplia para ser creíble, se procesan **todas** (fail-open) — nunca se recorta una selección para que quepa en un número. Un tipo sin `page_selection` no pasa por aquí. Spec: `catalogs/README.md`.
- **Camino A, paso 3**: extracción por segmento **paralela** (`ThreadPoolExecutor`, env var `EXTRACT_WORKERS`) vía `text_extractor.extract_document`, con el schema del catálogo.
- **Camino A, paso 4-5**: `normalize._recover_references` (regex) + `liquidacion.build_liquidacion_normalizada` + `qr_recovery` (si `QR_RECOVERY_ENABLED`), luego `engine.run_rules` / `resolve_fields` / `document_status`.
- **Camino A, paso 6**: `_run_cross_document_rules` — mismo pedimento, RFC importador, UUIDs únicos, conciliación de gastos/valores.
- El Camino B (`custom_validators/*.mjs`, `validator_module`) **no corre** para tipos field-centric como `PEDIMENTO_ADUANAL`; solo aplica a entradas que aún usan el DSL legacy.
- Tiempos medidos: 1 página ≈ 30 s · 27 páginas escaneadas ≈ 4 min · 86 páginas ≈ 6-8 min (límite duro de la Lambda: 15 min).

Recorrido con referencias exactas a archivo:línea, payloads y llamadas externas: **`TRAZA_CODIGO_COMPLETA.md`**.